MiniCPM5-2B 双国产卡推理吞吐优化技术报告

赛道：FlagOS 开放计算全球大赛（赛季二）赛道二：MiniCPM 模型推理吞吐性能优化

队伍：XLANG2026 | **队员**：邓乾（GitHub: xy200303，单人组队）

日期：2026-10-18 | **代码**：github.com/xy200303/minicpm-flagserve

1. 任务与总成绩

在官方指定 FlagOS 栈（vllm-plugin-FL `flagos-2026-s2` + FlagGems v5.3.5，vllm serve 参数与基线完全一致）上，对 MiniCPM5-2B 在两张国产卡上做有开发量的算子/内核/调度优化。全部改动落在提交代码内。

卡	场景	当日基线 total tok/s	最佳实测 total tok/s	提升
沐曦 C500 64GB	4k [4096,1024,64,256]	5089.645	**12529.39**	**+146.2%**
沐曦 C500 64GB	16k [16384,1024,64,128]	7029.675	**11088.13**	**+57.7%**
天数 BI-V150	4k	2028.01	**3778.65**	**+86.3%**
天数 BI-V150	16k	915.15	**3567.33**	**+289.8%**

> 绝对值的口径说明：BI-V150 这台共享开发实例的跨时段漂移极大（同一份代码同一天实测过 2346 与 3779 tok/s，差 60%），因此上表取"最佳实测日 + 当日基线"，**跨天不可比**；A20 之后的效果一律用同日交错 A/B 的相对收益表述（见 §4.8）：4k **+9.4%**、16k **+15.6%**，MATH-500 97.1%。

- 精度（MATH-500 Level 3，evalscope 官方口径，要求 ≥0.95）： C500 **97.1~98.1%**（采样自然波动区间）；BI-V150 **96.2~98.1%**（四次复核，均 ≥95%，基线 96.2%）。
- TTFT 双卡双场景均显著优于基线（无回退项）。详见各节。
- 天数成绩在 OnlineLab **32GB** 开发实例测得（评测机为 64GB）： 32GB 上 KV 池约 52 万 token，16k×64 并发需排队，绝对值低于 64GB 口径；本地复现基线为 1902.15 / 861.98（相对本地基线 +98.7% / +313.7%），同 silicon 下相对收益可直接迁移。

2. 统一优化方法论

两张卡共用同一套闭环，每一步都留有可复核的工件（探针脚本、profile 数据、单测、官方 benchmark 原始 CSV）：

1. **Profile 定位**：serve 时序日志 + torch profiler + host 侧 打点，先找到时间到底花在 GPU 内核、host 发射还是隐式同步。
2. **探针先行**：任何替换/新内核先在独立脚本里做全形状 bit-exact 或统计等价验证，再上 serve。

3. **Shape-aware dispatch**：对同一算子保留多个等价实现/执行 配置，按输入 shape 在派发层选择（赛题组 2026-10-10 确认合规）。
4. **官方口径验收**：只用官方 benchmark 脚本（4 跑弃首取后 3 均值）与 evalscope 精度评测，serve 参数一字未改。

3. 沐曦 C500：从采样链到 GEMM 路由（opt1-opt11）

3.1 sort-free 融合采样链（opt1/opt3/opt7/opt11）

问题：decode 采样尾部是纯 eager PyTorch：bf16→fp32 cast、

除温度、**全词表（130560）排序**做 top-p、gumbel-argmax，

每步 ~12ms 级别且全在 host 串行。

方案（自研 Triton 内核，FlagGems `fused/sampler_fused.py`、

`fused/top_p_sample.py`）：

- bf16 直读 lm_head 输出，温度折叠进 Gumbel-Max （argmax(x/T+g) ≡ argmax(x+T·g)，T<1e-5 退化为 argmax）；
- top-p 用**质量直方图 zoom**：单轮 1024-bin 直方图定位阈值 tau 所在 bin 并取其下沿，保留集质量恒 ≥ p·Z，**完全替代排序**；
- 词表维 split-parallel（32 splits × BLOCK 4096），小 batch 也能打满芯片。

结果：采样全链 12.4ms → 0.31ms/步（C500 4k 场景 +27%）。

正确性：greedy 与 torch.argmax 100% 一致；top-p 样本全部落在

核集（+直方图粒度 slack）内，与 stock 核采样 TV 距离在纯采样

噪声区间。

3.2 GEMM 布局与路由（opt2/opt4/opt6/opt8/opt11）

- 发现 MACA Triton `mm_kernel` 只支持 nn 布局且 prefill 比 vendor mcBLAS 慢 1.35-1.67x（126 组配置全扫仍不敌，vendor ~197TF 逼近峰值），而 FlagGems 进程级接管 aten::mm 后 vendor GEMM 从 Python 完全不可达。
- **ctypes 直绑 libmcblas `mcblasGemmEx`**（cuBLAS 兼容 ABI）， 注册 custom op 保证 compiled 区域安全；prefill 大 M 走 vendor， decode 细瘦 M 保持自研 Triton nn_db 单布局内核（权重只留一份，不多占显存）。
- 修复 `vocab_parallel_embedding` import 时绑死原版 `dispatch_unquantized_gemm` 导致 lm_head 从未进入优化路由的 问题：lm_head 603μs → 379μs。
- 4k +32.3%（opt8）是单步最大收益项。

3.3 host 隐式同步消除（opt10）

prefill trace GPU 空闲 18.1%，元凶之一是 vendor flash_attn 每层

每步 `prefill_seq_lens.tolist()`（DtoH 强同步 ×42 层）。改为纯

GPU cumsum 并缓存到当步 attn_metadata，42 层只算一次：

4k +10.9% / 16k +9.9%，TTFT 16k 20.8s → 18.0s。

3.4 C500 成绩阶梯（官方 benchmark 口径）

版本	4k tok/s	16k tok/s
官网基线	5089.645	7029.675
opt1-6	8135.29	8683.08
opt7 采样全融合	8273.58	8716.35
opt8 mcBLAS prefill	10946.86	10001.65
opt10 去同步	12145.31	10991.50
opt11（最终）	**12529.39**	**11088.13**

4. 天数 BI-V150：vendor FA 接入、发射配置重构与自研 decode 内核（A1–A20）

4.1 A1：prefill attention 切 vendor ixAttnBkd

问题：iluvatar 后端无 flash_attn 包，attention 落到 vLLM 通用 Triton 参考实现，16k prefill 每层每 2048-chunk 36.8ms，占 GPU busy 91.8%。

方案：接入 vendor ixAttnBkd 内核，确认其 causal 语义为 bottom-right（cmt=2）与 vLLM chunk prefill 对齐；内核级实测 vendor 在所有 kv 长度赢 12-20x，因此最终版**拆掉了 kv≥8192 阈值门**全量走 vendor。三个连环坑（per-layer tolist 同步、分配器风暴、阈值门控误判）逐一记录修复。

16k 861.98 → 2466.23（+186%）。

4.2 A3：builder host 隐式同步修复

`aten::index` 每步 2 次隐式阻塞 HtoD + 全流同步，占混合步 wall 57%。修复后混合步 wall 677→353ms、GPU busy 41%→76%。

4.3 A4：GEMM pinned-config（libtuner 误选根因）

发现 libtuner 运行时 autotune 在 event-timing 降级模式下**系统性误选** `mm_kernel` 配置。对 8 个 serve 形状离线全空间扫描后在派发层 pin 最优配置：decode GEMM 微基准 1.22-1.81x。

（SPLIT_K>1 路径补零初始化保证 atomic_add 正确性。）

4.4 A5：C500 采样融合内核跨平台移植

sort-free 采样链零芯片改动直接移植（后端无关 Triton）：

16k +1.90%，三跑极差 0.005%，收益确定。

4.5 A6：varlen vendor flash + Triton dual-gather（发射数砍半）

问题：prefill 段每请求 4 次发射（2×index_select + 1×aten attention + 1×copy-out），host 占混合步 wall 59%。

方案：

- 自研 Triton `_dual_gather_kernel`：利用 kv_cache 页内 K/V 连续布局，**一次发射**完成 K+V 分页收集（bit-exact， GPU 106→62μs）；
- 探针确认 `ixformer.flash_attn_varlen_func` 与 aten `_efficient_attention_forward`(cmt=2) 全形状 bit-exact 且支持 `out=` 直写，varlen 分支消掉 copy-out；
- cu_seqlens 在 builder 侧每步构建一次，42 层共享。

每请求发射 4→2，pre 段 host 1.77→0.13ms/call：

4k +5.83% / 16k +1.01%，TTFT 双降。

4.6 A7：decode attention 发射配置 pin（单步最大收益）

关键发现：stock Triton 默认 `num_warps=4, num_stages=3` 是 NVIDIA 数据中心甜点，在 BI-V150 上**系统性失配**——软件流水线（num_stages）在每个 warp 档上都是纯负收益，warps 越少越好。

离线扫描（含 3D 主内核 + reduce_segments 全链路）后仅对 3D flash-decoding 路径 pin `w1 s1`（TILE=16/SEG=16 经扫描本已最优）：

形状	stock w4s3	w1s1	加速
kv=16896, batch=64	6464μs	3524μs	**1.83x**
kv=4608, batch=64	1938μs	1099μs	**1.76x**

调度参数不改数学，输出与 stock **bit-exact（max_diff=0.0）**。

端到端 **4k +38.0% / 16k +39.4%**，16k TTFT 186.4s→137.0s。

此项同时说明 A2 的"decode 打平"结论只在默认配置下成立——**配置维度是独立于算法维度的优化空间**。

4.7 天数成绩阶梯

版本	4k tok/s	16k tok/s
本地基线（32GB）	1902.15	861.98
A1.5 vendor FA	2635.85	2466.23
A3 host 修复	2582.04	2488.58

A5 采样融合	2587.84	2532.68
A6 varlen+dual-gather	2738.65	2558.37
A7	3778.65	3567.33
A20（+ 自研 decode kernel）	当日 A/B **+9.4%**	当日 A/B **+15.6%**

（A4 端到端持平，收益预期在 64GB 评测机体现；各阶段精度均
97.1~98.1% 达标。）

4.8 A20：自研 paged decode attention 内核

A7 把 stock 3D decode kernel 的发射配置调优后，decode attention 仍是整步里
最大且唯一未压到带宽下限的一项。把 stock kernel 逐段拆开（同一 harness，
只替换被测 kernel 版本，B=64 / kv=4096 / 16q-2kv / d=128 / block=16 / bf16）：

版本	ms/层	有效带宽
只加载 K/V	0.52	509 GB/s
+ QK `tl.dot`	0.53	—
+ 在线 softmax	0.54	—
+ PV `tl.dot`（= stock 全量）	**0.90 – 1.04**	258 GB/s

即 **PV 那个 `tl.dot` 独占 0.36 ms**：它必须把 p 从 MMA 累加器布局转换成
A 操作数布局，每个 KV block 都要过一次共享内存往返；带宽和 softmax 都不是瓶颈。

据此重写内核（`decode_attn_fast.py`）：

1. **一个 CTA = 1 warp = 一个 (seq, kv_head, 段)**。`num_warps=1` 让 p 的布局 转换留在 warp 内（实测 w2 慢 16%，w4/w8 更差）。
2. **BLOCK_N=16**，p tile 不超过一个 MMA 形状（32/64 一致慢 12–30%）。
3. **base-2 在线 softmax**：`scale` 预先折算 `log2(e)`，`exp2` 走 MUFU； 用 `exp` 慢 10%。
4. **主循环不加 mask、只有 ragged 尾部单独一个带 mask 的 block**： 每块都 mask 会从 0.72 掉到 0.98 ms。
5. **段数固定、段长在 device 端由 `seq_lens` 现算**（与 stock 的 `num_par_softmax_segments` 同构）——这是能进 CUDA graph 的前提。

结果（隔离微基准 / 引擎内）：

	ms/层	带宽	对 stock
kv=4096	1.04 → **0.72**	258 → 372 GB/s	1.45x
kv=16384	3.50 → **2.41**	307 → 445 GB/s	1.45x

输出与 stock 的 max|diff| = 2.4e-4（ragged 长度 9.8e-4）。

两个 CUDA-graph 坑（都是靠 `VLLM_FL_DECODE_FAST_DEBUG=1` 的"首次拒绝原因"

日志定位的）：

- 段数一开始依赖 host 侧 max_seq_len，图捕获时它是 max_model_len=131072， nseg 超上限被拒绝 → 35 张图里全是 stock kernel，端到端零变化；
- 段缓存一开始在调用里 torch.empty，捕获期分配显存直接卡死第一个 graph； 改为 apply()（捕获之前）一次性预分配 slab。

混合步：vendor_flash_attn 会把 prefill+decode 混批拆成两条路径，
原先 decode 行仍走 stock。16k 用例绝大多数步都是混合步，所以只优化纯 decode 步
在 16k 上反而 −1.9%（decode 变快但调度/TTFT 劣化）。把 dec_meta 的 decode
切片接进自研内核后，16k 转正。

同日交错 A/B（官方用例）：

用例	stock	A20	变化
4k [4096,1024,64,256]	2239.43（A6b）	**2606.56**（B19，页对齐版）	**+16.4%**，TPOT 113.6→95.5 ms，TTFT −1.0%
16k [16384,1024,64,128]	2956.59（A7b）	**3541.34**（B20，页对齐版）	**+19.8%**，TTFT 155.5→130.1 s，TPOT 153.3→125.6 ms

精度：MATH-500 Level 3 四次复核 **97.1% / 98.1% / 96.2% / 97.1%**（105 题，基线 96.2%），
均 ≥95%；跑间差异来自采样（temperature 1.0 未固定 seed）与段内 fp32 求和顺序的等价扰动
（页对齐与通用分段的 greedy 对拍 4/5 完全一致，第 5 条从 ~30 token 起换措辞）。
可审计性：混合步的 decode 切片每 4000 次调用打一行
decode_attn_fast slice: used=X skipped=Y。16k 两次实测均为
used=44000, skipped=0，即 decode 切片 100% 命中自研内核，不存在"偶尔退回 stock"。
为什么停在 372 GB/s：把 Triton 编出的 Iluvatar 汇编 dump 出来可以看到，
卡上**有 bf16 的 ml_matrix_mad_f32_bf16_m16n16k16**（fp32 累加），QK 与 PV 两个
dot 都落在它上面；代价不在算力，而在 **MMA 累加器布局 → 点积操作数布局的转换**
（TTGIR 里是一条 ttg.convert_layout : #mma -> #ttg.dot_op，汇编里展开成
ml_lsa_blkload_x1×72 / ml_slb_load_b32x1×48 / ml_lsa_store_a64_dword×32 /
ml_slb_shuffle_rtn_b32×22 / ml_byte_permute_b32×25 与 6 处 sl_barrier）。
实测把 K/V 换成 fp16 并不更快（0.699 vs 0.698 ms/层），确认瓶颈不在 dtype，
而在 MMA 前后的布局搬运。
天花板测量：把 attention 整个跳过（探针）后，4k/B=64 的 decode 步是
11.7 ms；A20 是 43.7 ms，stock 是 57.1 ms。即 attention 现为 32 ms/步，
而 KV 读的硬件下限是 19.3 ms（11.3 GB @ 587 GB/s）。剩余 12 ms 全部卡在
PV 的 MMA 转换路径上：当前 372 GB/s 已是可达负载带宽（505–631 GB/s）的

~70%，再往上就需要脱离 Triton。

5. 创新点

1. **跨双平台的 shape-aware dispatch 统一框架**：同一套"离线 全空间扫描 + 派发层按 shape 选实现/配置"方法论落在两种 国产芯片上，覆盖 GEMM、attention、采样三大类算子，且有 赛题组明确合规口径。
2. **sort-free top-p 采样**：1024-bin 质量直方图 zoom 替代全词表 排序，保留集质量恒 ≥ p·Z（有下界保证，非近似猜阈值）， 双平台复用，采样链 ~40x。
3. **发射数削减范式**：dual-gather（页内 K/V 连续布局利用）+ varlen `out=` 直写，把 attention prefill 的 host 开销从 "4 发射/请求"降到 2，全部 bit-exact 探针验证。
4. **配置维度系统性诊断**：NVIDIA 默认启发式（w4s3）与 libtuner event-timing 降级在国产芯片上的失配根因分析， 单此一项带来 4k/16k 各 ~40% 端到端收益。
5. **自研 decode attention 内核**：从"逐段拆解定位到 PV 的 MMA 布局转换" 出发，用单 warp / 小 tile / base-2 softmax / 尾部 mask 四项改动把 decode attention 从 258 GB/s 提到 372 GB/s（4k +9.4%、16k +15.6%，同日交错 A/B）， 并用"跳过 attention 的探针"量出该方向的天花板（19.3 ms/步）。
6. **零精度妥协**：所有性能改动要么 bit-exact（A6/A7），要么 统计等价且 MATH-500 L3 实测 ≥97.1%（采样融合、GEMM 路由）。

6. 阴性结果与探索覆盖度

- **A2**：自研 Triton flash-decoding 与 stock 打平（默认配置下）， 转投配置维度后由 A7 收获。
- **opt9**：C500 attention 内核选择全空间扫描确认已到 vendor 上限。
- **A8a**：dec host 壳子收益 <1%（混合步占比 1.5-6% 且 GPU-bound）， 算账否决。
- **A8b**：lm_head GEMM 48 组配置扫描，aten vendor 已贴带宽墙， 无可 pin。
- **A8c**：采样内核 warps/splits 重扫，现配置即甜点。
- **A9**：n-gram 投机采样探针（benchmark 同分布 arange prompt）： 75% 请求零命中，聚合命中系单条退化复读序列的假象，期望收益 <10% 且集成复杂度高，否决。
- **A20-int8 KV**：int8 KV（per-token-head scale）在 BI-V150 上**实测否决**： 只加载就 1.06 ms/层 vs bf16 0.72（kv=4096），kv=16384 是 3.63 vs 2.41。 瓶颈不是反量化 ALU，而是这套 Triton 后端对 1 字节 gather 的吞吐只有 2 字节的 1/3（每列仅 128 B 连续）。
- **A20-paged prefill**：想用 `ixformer.batch_paged_attention` 省掉 prefill 的逐层反分页 gather，但该 API 要求 cache 首维**恰好**等于 `max_page_num*page_size`，与 vLLM 全局池（32603 页）和 `block_tables` 宽度（8192）不兼容；tensor-desc 路径直接 `CUINFER_STATUS_BAD_PARAM`。 **事后用 CUDA 事件更正了收益判断**：gather 每次仅 0.03–0.05 ms，而 vendor FA 每次 1.7–3.3 ms，gather 只占 prefill GPU 时间的 **~1.5%**——即便打通也只有 1–2% 收益。16k 的 prefill 是 FA 计算受限（有效算力 ~115 TFLOPS，接近本卡上限）。
- **A21-IR-RMSNorm**：把 RMSNorm 从 vLLM 的 IR "native"（纯 torch 链）换成自研 Triton 融合核， 隔离里逐位一致且 2.5–7.2x（N=64: 132.8→54.0 μs；N=8192: 2039→281 μs），但引擎内 **−5.9%**、TTFT 翻倍。原因：IR op 在编译期被 inline，核只能包成 opaque `torch.library.custom_op`，inductor 因此失去对相邻算子的融合（prefill 最敏感）。已回退。

- **量化**评估（权重侧）：decode 权重带宽 ~5.3 GB/步已贴带宽墙，理论上 W8/W4 可省 5–7 ms/步；但本卡 Triton matmul 比厂商慢 6 倍（FlagGems matmul 0.42 vs vendor 0.069 ms），且 1 字节加载吞吐已被 int8-KV 实验 证伪，故未投入。

7. 合规对照

约束	落实
serve 参数与基线一致	全部成绩均在原命令下测得（天数含 FULL_DECODE_ONLY）
禁用其他模型/自训 draft	未使用；n-gram 投机经探针后主动放弃
禁止超参开量化/投机	未开启；采样/量化相关工作均为自研内核集成
禁止调 benchmark	官方脚本原样，所有 CSV 留档
不删框架算子选择逻辑	所有 pin/路由均在插件派发层内新增，stock 路径保留，含 A/B 开关
策略须落在提交代码	报告每项均对应 vllm-plugin-FL / FlagGems 内具体 commit

8. 复现指南

- 仓库：github.com/xy200303/minicpm-flagserve
- `vllm-plugin-FL/iluvatar-vendor-fa`：天数补丁（A1–A7，至 3f05c36d）
- A20 自研 decode 内核 + 混合步 hook：`vllm-plugin-FL@84abed0` + `docs/experiments/a20/apply_a20.py` （干净 clone 应用后与线上三个文件字节一致）
- `vllm-plugin-FL/dev`：C500 补丁（opt1-opt11，至 f4971e7e）
- `FlagGems/dev`：采样融合内核、GEMM pin 等（至 5eac6b39， 与天数机器上的文件逐字节一致）
- `main`：本文档及全部基线/优化记录（docs/baselines/**）
- serve / benchmark / evalscope 命令与官网完全一致，见 `docs/baselines/*/README.md`。
- 所有 A/B 开关：`VLLM_FL_DISABLE_ILUVATAR_VENDOR_FA`、 `VLLM_FL_DISABLE_VARLEN_PREFILL`、`VLLM_FL_DISABLE_DUAL_GATHER`、 `VLLM_FL_DISABLE_DECODE_PIN`、`VLLM_FL_DISABLE_SAMPLER_FUSION`。

初稿待办

- [ ] 双卡最终精度统一复测口径描述（97.1~98.1 波动说明）
- [ ] 插图：成绩阶梯图 ×2、decode 发射配置扫描热力图、采样链结构图
- [ ] 转 report.pdf（word-skills 排版）